T3000 API & Protocol

Message: GET_ENTRIES · built Nov 13, 2026

Message: GET_ENTRIES

GET_ENTRIES retrieves multiple specific entries by their exact indices. Unlike GET_WEBVIEW_LIST which reads a continuous range, this message fetches scattered individual entries.

Common Uses:

- Reading specific points (e.g., inputs 0, 5, 12)
- Batch retrieval of non-sequential data
- Updating specific dashboard widgets
- Custom point monitoring

Overview

Message Type: GET_ENTRIES **File Location:** `BacnetWebView.cpp` line 2550 **Purpose:** Read specific entries by exact indices

Request Format

JSON

```
{
  "action": "GET_ENTRIES",
  "source": 1,
  "data": [
    {
      "panelId": 0,
      "index": 0,
      "type": 1
    },
    {
      "panelId": 0,
      "index": 5,
      "type": 1
    },
    {
      "panelId": 0,
      "index": 12,
      "type": 2
    }
  ]
}
```

Parameters

Parameter	Type	Required	Description
`action`	String	✅	Must be "GET_ENTRIES"
`source`	Integer	✅	Message source: 0=T3000, 1=WebUI
`data`	Array	✅	Array of entry specifications

Entry Specification

Field	Type	Required	Description
`panelId`	Integer	✅	Panel number
`index`	Integer	✅	Entry index
`type`	Integer	✅	Entry type (1=IN, 2=OUT, 3=VAR, etc.)

Response Format

JSON


```
{
  "action": "GET_ENTRIES_RES",
  "data": [
    {
      "panelId": 0,
      "type": "INPUT",
      "index": 0,
      "description": "Room Temp",
      "label": "RM_TEMP",
      "value": 720,
      "range": 1
    },
    {
      "panelId": 0,
      "type": "INPUT",
      "index": 5,
      "description": "Outside Temp",
      "label": "OUT_TEMP",
      "value": 450,
      "range": 1
    },
    {
      "panelId": 0,
      "type": "OUTPUT",
      "index": 12,
      "description": "Cooling Valve",
      "label": "COOL_VLV",
      "value": 500,
      "range": 0
    }
  ]
}
```

Implementation

C++

```
case WEBVIEW_MESSAGE_TYPE::GET_ENTRIES:
{
    tempjson["action"] = "GET_ENTRIES_RES";
    Json::Value data = json["data"];

    if (data.size())
    {
        for (int i = 0; i < data.size(); i++)
        {
            Json::Value entry = data[i];
            int npanel_id = entry.get("panelId", -1).asInt();
            int entry_index = entry.get("index", -1).asInt();
            int entry_type = entry.get("type", -1).asInt();

            switch (entry_type)
            {
                case BAC_IN:
                    tempjson["data"][i]["type"] = "INPUT";
                    tempjson["data"][i]["description"] =
                        (char*)g_Input_data[npanel_id].at(entry_index).description;
                    tempjson["data"][i]["value"] =
                        g_Input_data[npanel_id].at(entry_index).value;
                    // ... more fields ...
                    break;

                case BAC_OUT:
                    tempjson["data"][i]["type"] = "OUTPUT";
                    // ... populate fields ...
                    break;

                // ... other types ...
            }
        }
    }

    break;
}
```

Performance

Entries	Time
1-5	< 50ms
10-20	50-150ms
50+	150-500ms

Usage Examples

Example 1: Dashboard Widgets

Read specific points for dashboard display:

JSON

```
{
  "action": "GET_ENTRIES",
  "source": 1,
  "data": [
    { "panelId": 0, "index": 0, "type": 1 },
    { "panelId": 0, "index": 1, "type": 1 },
    { "panelId": 0, "index": 0, "type": 2 },
    { "panelId": 0, "index": 1, "type": 2 }
  ]
}
```

Example 2: Mixed Types

Read different entry types in one request:

JSON

```
{
  "action": "GET_ENTRIES",
  "source": 1,
  "data": [
    { "panelId": 0, "index": 5, "type": 1 },
    { "panelId": 0, "index": 3, "type": 2 },
    { "panelId": 0, "index": 10, "type": 3 },
    { "panelId": 0, "index": 0, "type": 6 }
  ]
}
```

Example 3: Multiple Panels

Read from different panels:

JSON

```
{
  "action": "GET_ENTRIES",
  "source": 1,
  "data": [
    { "panelId": 0, "index": 0, "type": 1 },
    { "panelId": 1, "index": 0, "type": 1 },
    { "panelId": 2, "index": 0, "type": 1 }
  ]
}
```

Comparison: GET_ENTRIES vs GET_WEBVIEW_LIST

Feature	GET_ENTRIES	GET_WEBVIEW_LIST
Selection	Specific indices	Continuous range
Types	Mixed types allowed	Single type only
Best for	Scattered points	Sequential ranges
Request size	Smaller (specific)	Larger (range)
Use case	Dashboards, widgets	Full list views

Use GET_ENTRIES when:

- You need specific scattered points
- Different entry types in one request
- Dashboard widgets with custom point selections

Use GET_WEBVIEW_LIST when:

- You need a continuous range
- Loading full lists (all inputs, all outputs)
- Single entry type operations

Error Handling

JSON

```
{
  "action": "ERROR_RES",
  "error": {
    "code": -2,
    "message": "Invalid entry",
    "details": "Entry not found: panel=0, type=1, index=99"
  }
}
```

See Also

- GET_WEBVIEW_LIST - Read continuous ranges
- GET_PANEL_DATA - Load all panel data
- Control Messages Index